서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험Ⅱ

# STM32_0 실험 예비 보고서

STM32_0 LED 와 버튼

학과 전자전기컴퓨터공학부

이름 / 학번 주윤찬 / 2023440132

작성일 2026 년 10 월 12 일

대상 LED·버튼 폴링과 FPGA 조합회로

실습환경 NUCLEO-G474RE / Combo II-DLD

## 1. 실험 목표와 준비

2026 년 10 월 12 일 실습에서는 NUCLEO-G474RE 의 LED 와 버튼을 폴링 방식으로 제어하고, Combo II-DLD 에서 4 비트 곱셈기와 BCD·이진수 변환 회로를 확인한다. 제공 예제 2 개를 각각 실행한 뒤 STM32 과제 1·2 와 FPGA 실습 2 개를 순서대로 수행한다. [1, 2]

| 구분 | 수행할 내용 |
|---|---|
| 제공 예제 1 | PA5 LED 를 1 초 간격으로 변경 |
| 제공 예제 2 | PC13 버튼 입력으로 LED 상태 변경 |
| 직접 작성 과제 | 내장 LED 간격 변경 → 외부 LED 적용 |
| FPGA 실습 | 4 비트 곱셈기 → BCD·이진수 변환 |

### 실험 환경

NUCLEO-G474RE, ST-LINK USB 케이블, 외부 LED, 330 Ω~1 kΩ 저항, 브레드보드와 연결선을 준비한다. FPGA 실습에는 Combo II-DLD, 장비 교재와 핀맵, Vivado 를 사용한다. STM32 프로그램은 VS Code 와 STM32CubeIDE for Visual Studio Code 환경에서 빌드한다. [1, 2]

수업 기준 프로젝트 Glaysia/ece2_474 의 v1.1.0 태그를 받아 stm32_lab 에 준비하였다. 원본 ece2_474_routine.c 는 original 폴더에 보관하였다. 아래 명령은 프로젝트를 처음 받을 때의 참고이며, 실습에서는 준비된 ece2_474.code-workspace 를 연다. [1, 2]

```
git clone --branch v1.1.0 --depth 1 `
  https://github.com/Glaysia/ece2_474.git `
  stm32_lab
code .\stm32_lab\ece2_474.code-workspace
```

위 명령의 줄 끝 `는 PowerShell 줄 연결 문자다. STM32 공식 도구 설치 후 VS Code 에서 Configure → Build 를 확인한다. JP5 의 ST-LINK USB 전원 위치와 USB 연결을 확인하고 F5 로 다운로드한다. main 에서 정지하면 F5 를 다시 눌러 실행한다. 새 코드는 Build 또는 RESET 만으로 보드에 기록되지 않는다. [1, 2]

### 예제 전환과 코드 보존

준비한 코드에서는 Core/Inc/stm32_0_config.h 의 STM32_0_STAGE 값으로 예제를 선택한다. 1 은 제공 예제 1, 2 는 제공 예제 2, 3 은 과제 1, 4 는 과제 2 이며 현재 값은 1 이다. 디버깅을 종료하고 값을 변경·저장한 뒤 Build 와 F5 다운로드를 다시 수행한다. polling_routine()은 하나이며 선택된 분기만 컴파일된다. 타이머·PWM·UART 시작 호출은 주석 상태이고 버튼 ISR 과 HAL 콜백 연결은 유지하였다.

## 2. 기초 원리와 제공 예제

### MCU 제어와 GPIO

FPGA 는 Verilog 로 회로를 구성하는 반면 STM32 는 CPU 가 C 에서 컴파일된 기계어를 실행하여 주변장치를 제어한다. C 소스는 컴파일·링크를 거쳐 ELF 가 되고 ST-LINK/SWD 로 Flash 에 기록된다. GPIO 함수는 주변장치 레지스터를 통해 핀의 상태를 읽거나 출력 전압을 변경한다. [1, 3]

GPIO_PinState 는 GPIO_PIN_RESET 과 GPIO_PIN_SET 을 사용하는 열거형이다. RESET 은 핀 LOW, SET 은 HIGH 를 뜻하며 보드 재시작 버튼과 다르다. 이 교안의 PA5 LED 는 HIGH 에서 켜지고 PC13 사용자 버튼은 눌렀을 때 HIGH 로 판정한다. uint32_t 는 부호 없는 32 비트 정수이고 static 지역 변수는 함수 호출 사이에 값을 유지한다. [2]

HAL_GPIO_ReadPin 은 입력 레지스터 IDR 을 확인한다. WritePin 은 지정한 값을 출력에 반영하고 TogglePin 은 출력 상태를 반전한다. 따라서 SET 을 반복 호출해도 LED 는 켜진 상태를 유지하지만 Toggle 은 호출마다 켜짐·꺼짐이 바뀐다. 이 해석은 ST 의 GPIO 드라이버 구현을 근거로 한다. [3]

### 제공 예제 1 의 수행 계획

HAL_GetTick()으로 현재 ms 시간을 읽고 now_ms - last_ms ≥ 1000 이면 LED 상태를 반전하여 PA5 에 출력한다. 변화 시각을 last_ms 에 저장하고 그 밖의 호출에서는 출력을 유지한다. 긴 HAL_Delay 없이 메인 루프를 반복하는 비차단 방식이다. [2]

예상 동작은 초기 OFF 이후 약 1 초마다 ON/OFF 가 번갈아 나타나는 것이다. ON 1 초와 OFF 1 초를 합친 전체 주기는 2 초이며 반복 주파수는 0.5 Hz 다.

영상이나 시간 기록에서 연속된 상태 변경 시각을 비교하여 간격을 확인할 계획이다.

### 제공 예제 2 의 수행 계획

current 가 SET 이고 previous 가 RESET 인 상승 에지만 버튼 누름으로 판정한다. 마지막 유효 입력 이후 200 ms 이상 지났을 때 LED 를 한 번 토글하고, 매 호출 끝에 previous 를 current 로 갱신한다. 200 ms 이내 반복 입력은 무시한다. [2]

짧게 누르기, 길게 누르기, 떼었다 다시 누르기를 구분하여 확인한다. 길게 누르면 current 와 previous 가 모두 SET 이 되므로 LED 가 계속 토글되지 않아야 한다. 200 ms 제한은 간단한 중복 검출 억제이며 버튼 입력이 일정 시간 안정되었음을 보장하는 완전한 디바운스와는 구별한다.

## 3. STM32_0 과제 1 내장 LED

### 설계 목표와 처리 순서

버튼을 누를 때마다 LED 상태 변경 간격을 1000 → 2000 → 4000 → 8000 → 1000 ms 로 순환시킨다. 버튼이 없을 때도 선택한 간격으로 LED 가 계속 점멸하고, 길게 누를 때 간격은 한 번만 바뀌어야 한다. 버튼 검출과 시간 비교를 같은 polling_routine() 안에서 독립적으로 처리한다. [2]

간격은 interval_ms 에 저장한다. 유효한 상승 에지마다 8000 ms 이면 1000 ms 로 돌아가고, 그 외에는 두 배로 변경한다. 본 설계에서는 간격을 바꾼 시점부터 새 대기 시간을 시작하기 위해 last_change_ms 도 현재 시각으로 갱신한다. 버튼 자체는 LED 를 즉시 반전하지 않는다. 이 정책은 실험 영상의 예상 동작과 비교할 기준이다.

### 준비한 과제 1 의 핵심 동작

```
void polling_routine(void)
{
  static GPIO_PinState prev = GPIO_PIN_RESET;
  static uint32_t press_ms = 0U;
  static uint32_t change_ms = 0U;
  static uint32_t interval_ms = 1000U;
  uint32_t now = HAL_GetTick();
  GPIO_PinState cur = HAL_GPIO_ReadPin(
    BUTTON_ece2_GPIO_Port, BUTTON_ece2_Pin);

  if (cur == GPIO_PIN_SET &&
      prev == GPIO_PIN_RESET &&
      (uint32_t)(now - press_ms) >= 200U) {
    press_ms = now;
    interval_ms = (interval_ms == 8000U)
                  ? 1000U : interval_ms * 2U;
    change_ms = now;
  }
  prev = cur;

  if ((uint32_t)(now - change_ms) >=
      interval_ms) {
    change_ms = now;
    HAL_GPIO_TogglePin(LED_ece2_GPIO_Port,
                       LED_ece2_Pin);
  }
}
```

위 코드는 저장된 STAGE 3 의 동작을 설명하는 발췌이며, 지면상 previous·last_press_ms·last_change_ms·now_ms·current 를 각각 prev·press_ms·change_ms·now·cur 로 줄여 표기하였다. 실제 코드는 ece2_474_routine.c 에 있다. 네 단계 모두 ARM 빌드를 통과했으며 보드 동작은 실험 중 확인한다. 초기 200 ms 의 입력은 제한되고, 디버거 중단 등 긴 지연에서 놓친 토글을 한꺼번에 복구하지 않는다.

| 상태 변경 간격 | 전체 주기 / 반복 주파수 |
|---|---|
| 1 초 | 2 초 / 0.5 Hz |
| 2 초 | 4 초 / 0.25 Hz |
| 4 초 | 8 초 / 0.125 Hz |
| 8 초 | 16 초 / 0.0625 Hz |

### 검증 계획

리셋 후 기본 1 초 간격을 확인하고 버튼 4 회를 충분히 떨어뜨려 눌러 2·4·8·1 초의 순환을 기록한다. 각 단계에서 최소 2 회 상태 변화를 촬영한다. 버튼을 2 초 이상 유지해 단계가 추가로 바뀌지 않는지 확인하고, 200 ms 이내 빠른 재입력이 억제되는지도 확인한다. 간격 변경 순간의 출력 유지와 새 간격의 시작 시각을 함께 기록한다.

## 4. STM32_0 과제 2 외부 LED

### 외부 LED 연결과 코드 변경

과제 1 의 내장 LED 동작을 먼저 확인하고 코드를 보존한다. 과제 2 는 PB0 에 연결한 외부 LED 를 사용한다. PB0 → 330 Ω~1 kΩ 저항 → LED 긴 다리(양극), LED 짧은 다리(음극) → GND 로 연결한다. 연결을 바꿀 때는 전원을 끄고 극성과 공통 접지를 확인한다. [2]

| 항목 | 과제 2 에서 사용할 이름 |
|---|---|
| 출력 핀 | PB0 |
| 핀 매크로 | GPIO_OUT1_ece2_Pin |
| 포트 매크로 | GPIO_OUT1_ece2_GPIO_Port |
| 대안 | PB1 / GPIO_OUT2_ece2 |

실제 준비 코드에서는 STM32_0_STAGE 를 4 로 선택하면 외부 LED 로 출력이 바뀐다. STM32_0_EXTERNAL_PIN 은 1 이면 PB0, 2 이면 PB1 이며 기본값은 1 이다. 아래는 PB0 분기의 호출이다. 버튼 판정과 간격 순환은 과제 1 과 동일하다. 핀 선택을 바꾸면 배선도 함께 변경하고 다시 빌드·다운로드한다.

```
HAL_GPIO_TogglePin(
  GPIO_OUT1_ece2_GPIO_Port,
  GPIO_OUT1_ece2_Pin);
```

### 전류 제한의 원리

직렬 저항은 LED 에 흐르는 전류를 제한한다. 3.3 V 출력과 LED 순방향 전압 2.0 V 를 가정하면 I ≈ (3.3 - 2.0)/R 이다. 330 Ω 에서는 약 3.94 mA, 1 kΩ 에서는 약 1.30 mA 다. 이 값은 LED 사양과 실제 출력 전압에 따라 달라지는 계산 예시이며 측정값이 아니다. 실습에서는 교안이 지정한 저항 범위를 사용한다. [2]

### 예상 결과와 확인 계획

HIGH 출력 때 외부 LED 가 켜지고 LOW 때 꺼지며, 버튼 누름마다 1·2·4·8·1 초 간격으로 바뀔 것으로 예상한다. 단순히 외부 LED 가 켜지는 것만 확인하지 않고 모든 간격과 길게 누르기를 다시 검사한다. 영상에는 PB0 배선, 저항, LED 극성이 보이도록 촬영하고 내장 LED 과제와 외부 LED 과제의 자료를 분리한다.

출력이 변하는데 LED 가 켜지지 않으면 LED 극성, GND, 저항과 접촉 상태를 확인한다. 내장 LED 만 변하면 STAGE=4 선택과 새 펌웨어 다운로드 여부를 확인한다. 단계가 반복 변경되면 버튼의 접점 변화와 previous 상태 갱신을 점검한다.

## 5. FPGA 4 비트 곱셈기

### 회로 구조와 예상 출력

최상위 mult_4bit 는 두 4 비트 unsigned 입력 a·b 와 8 비트 출력 m 을 연결하고, multiply_unsigned 하위 모듈이 실제 곱셈을 수행한다. 조합회로이므로 별도의 클록이나 저장 상태 없이 입력 변화에 따라 출력이 계산된다. 입력 범위는 각각 0~15 이며 최대 결과 15 × 15 = 225 는 8 비트로 표현할 수 있다. [2]

```
module mult_4bit(
  input wire [3:0] a, b,
  output wire [7:0] m);
  multiply_unsigned #(.WIDTH(4)) u_multiply(
    .a(a), .b(b), .product(m));
endmodule


module multiply_unsigned #(parameter WIDTH=4)(
  input wire [WIDTH-1:0] a, b,
  output wire [2*WIDTH-1:0] product);
  assign product = a * b;
endmodule
```

위 회로는 매뉴얼의 최상위·하위 모듈 연결을 옮긴 핵심 코드다. unsigned 폭과 최상위 이름을 유지하고 설계 소스에 두 모듈을 모두 추가한다. TB 는 시뮬레이션 소스로만 등록한다. [2]

| 입력 a × b | 예상 m (10 진 / 16 진) |
|---|---|
| 0 × 15 | 0 / 00 |
| 3 × 5 | 15 / 0F |
| 8 × 8 | 64 / 40 |
| 15 × 15 | 225 / E1 |

### 시뮬레이션과 보드 수행 계획

준비한 시뮬레이션 최상위는 tb_mult_4bit 다. x 와 y 를 0~15 까지 반복하여 256 개 조합을 넣고, 입력 변경 후 1 ns 를 기다려 m !== x*y 이면 오류로 종료한다. 사전 XSim 실행에서 PASS multiplier: 256 input pairs 를 확인했고 합성도 완료하였다.
실험실에서는 같은 TB 를 재실행하고 파형을 저장한다.

보드 시연은 준비된 mult_4bit.xpr 에서 진행한다. 설계 최상위는 mult_4bit, 입력 a·b 는 DIP1~4·DIP5~8 에 대응하며 출력은 8 개 LED 를 사용한다. 기존 LAB1·LAB3 의 Combo II-DLD 핀 배정을 pins.xdc 에 반영하였다. 실제 장비가 xc7s75fgga484-1 인지 확인한 뒤 Implementation, Bitstream 생성과 보드 다운로드를 수행한다. 1 ns 의 TB 대기는 실물 전파 지연 측정값이 아니다.

## 6. FPGA BCD·이진수 변환

### 회로 구조와 유효 입력

BCD 는 십진 숫자 한 자리를 4 비트로 표현한다. 십의 자리 ten 과 일의 자리 one 이 각각 0~9 일 때 binary = 10 × ten + one 으로 변환한다. 최대값 99 는 7 비트면 충분하며 $1100011_2$이다. 매뉴얼 회로는 valid 를 추가하여 각 자리 중 하나라도 10~15 이면 0 으로 판정한다. [2]

```
module bcd_to_binary(
  input wire [3:0] ten, one,
  output wire [6:0] binary,
  output wire valid);
  assign binary = ten * 7'd10 + one;
  assign valid = (ten <= 9) && (one <= 9);
endmodule


module bcd_conv(
  input wire [3:0] one, ten,
  output wire [6:0] bin,
  output wire valid);
  bcd_to_binary u_convert(.ten(ten),
    .one(one), .binary(bin), .valid(valid));
endmodule
```

위 코드는 준비한 변환 모듈의 핵심이다. 최상위 입력 선언은 one, ten 순서지만, 하위 모듈과는 .ten(ten) 등의 이름으로 연결하여 순서 혼동을 줄였다. invalid 입력의 bin 은 십진수 변환 결과로 해석하지 않고 valid=0 을 우선 확인한다.

| ten / one | 예상 bin / valid |
|---|---|
| 0 / 0 | 0000000 / 1 |
| 1 / 2 | 0001100 (12) / 1 |
| 9 / 9 | 1100011 (99) / 1 |
| 10 / 0 | 변환값 판정 제외 / 0 |

### 검증 계획

준비한 시뮬레이션 최상위는 tb_bcd_conv 다. 각 자리의 0~15 를 전수 검사하며, 유효한 100 개는 valid=1 및 bin=10×ten+one, 나머지 156 개는 valid=0 을 검사한다. 사전 XSim 실행에서 PASS BCD: 100 valid and 156 invalid inputs 를 확인했고 합성도 완료하였다. 두 회로는 별도 Vivado 프로젝트로 저장하였다.

보드 시연은 준비된 bcd_conv.xpr 에서 진행한다. ten 은 DIP1~4, one 은 DIP5~8 에 연결되며, bin[6:0]과 valid 를 각각 LED 에 배정한 pins.xdc 를 사용한다. 같은 Combo II-DLD 장비인지 확인 후 Implementation 과 Bitstream 생성·다운로드를 진행한다. 00·09·10·12·99 및 invalid 조합을 입력해 자리 순서와 LSB/MSB 를 확인한다.

## 7. 기록 계획과 예상 결론

### 실험 중 확보할 자료

STM32 제공 예제와 과제 1·2 의 실행 코드를 구분해 보관하고, 내장·외부 LED 의 간격 순환과 길게 누르기를 영상으로 기록한다. 각 단계의 연속된 변화 시각 $t_1$·$t_2$에서 $\Delta t = t_2 - t_1$을 구하고, 목표 간격과의 오차율 $|\Delta t - \Delta t_{목표}| / \Delta t_{목표} \times 100\%$를 계산한다. 영상 프레임 간격과 사람이 누르는 시각 때문에 측정 정밀도에 제한이 있다.

사전 준비에서는 STM32 네 단계의 ARM 빌드, FPGA 두 회로의 기능 시뮬레이션과 합성을 확인하였다. ELF·HEX 는 firmware, 검증 로그는 verification 폴더에 보관하였다. 실제 보드 다운로드·배선·점멸 시간 측정과 FPGA Implementation·Bitstream 생성은 아직 수행하지 않았다. 본문의 예상 출력 표는 실물 측정 결과가 아니며, 실험 후 파형과 보드 영상을 추가한다.

### 저장소와 제출 버전 관리

실험 후에는 개인 GitHub 저장소에서 전체 소스와 시연 자료를 확인할 수 있도록 구성하고 저장소 URL, 제출 태그와 최종 commit hash 를 기록한다. 수업 기준 v1.1.0 은 받아오는 프로젝트의 태그이며 개인 제출 태그와 구별한다. 이번 예비보고서에서는 아직 생성하지 않은 개인 저장소 주소나 커밋 해시를 기재하지 않는다. [1]

### 예상 결론

폴링을 이용해 버튼 검출과 시간 기반 출력을 한 루프에서 처리하도록 구현하였다. 상승 에지 판정으로 길게 누름의 반복 처리를 제한하고, 간격 변수를 순환시켜 내장·외부 LED 에 같은 알고리즘을 적용한다. FPGA 의 unsigned 비트 폭과 BCD 유효 조건은 사전 전수 시뮬레이션에서 확인하였다. 실험에서는 LED 의 실제 점멸 간격과 보드의 입력·출력 일치를 확인할 계획이다.

### 참고문헌

[1] 이해리,「STM32 실습 안내」, 2026.10.06., STM32_4week_plan.pdf, 2~8·12 쪽. 실습일 2026.10.12.

[2] thechef030, STM32_0 실습 매뉴얼, stm32-0 브랜치. docs/index.md, docs/STM32_0.md, docs/FPGA_additional.md 및 README.md. https://github.com/thechef030/STM32_0/tree/stm32-0